Proving Basic Algorithms Correct: Sorting

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Computation Induction

So far: *structural induction*, since all functions were primitive recursive.

But `merge` shortens `xs` in one recursive call and `ys` in the other.

©Mohammad Abdulaziz

Computation Induction: Example

```
merge [] ys = ys
merge xs [] = xs
merge (x # xs) (y # ys) =
  (if x ≤ y then x # merge xs (y # ys)
   else y # merge (x # xs) ys)
```

Computation Induction: Example

```
merge [] ys = ys
merge xs [] = xs
merge (x # xs) (y # ys) =
  (if x ≤ y then x # merge xs (y # ys)
   else y # merge (x # xs) ys)
```

Prove `length (merge xs ys) = length xs + length ys`
by `merge.induct`, with `P xs ys` this property: show

1. `P [] ys`
2. `P (x # xs) []`
3. `(x ≤ y ⟹ P xs (y # ys)) ⟹`
   `(¬ x ≤ y ⟹ P (x # xs) ys) ⟹`
   `P (x # xs) (y # ys)`

©Mohammad Abdulaziz

Computation Induction: Example

```
merge [] ys = ys
merge xs [] = xs
merge (x # xs) (y # ys) =
  (if x ≤ y then x # merge xs (y # ys)
   else y # merge (x # xs) ys)
```

Prove `length (merge xs ys) = length xs + length ys`
by `merge.induct`, with `P xs ys` this property: show

1. `P [] ys`
2. `P (x # xs) []`
3. `(x ≤ y ⟹ P xs (y # ys)) ⟹`
   `(¬ x ≤ y ⟹ P (x # xs) ys) ⟹`
   `P (x # xs) (y # ys)`

In Isabelle: `proof (induction xs ys rule: merge.induct)`
Cases are named `1`, `2`, `3`; the IH of case 3 is `"3.IH"` (with quotes).

©Mohammad Abdulaziz

Informal Proof: Length of `merge`

Lemma

```
length (merge xs ys) = length xs + length ys
```

©Mohammad Abdulaziz